

```
=====
Python OOPS
=====
```

```
** OOPS = Object Oriented Programming System

** OOPS is a programming approach where we design software using classes and objects.

** In real life, everything can be treated as an object.
```

```
Student -----> name, email, phno, course, fee
```

```
Employee -----> id, name, salary, project
```

```
Bank Account ----> account number, balance, branch name, deposit, withdraw
```

```
** Without OOPS, we usually write multiple variables and functions separately like below...
```

```
-----
```

```
student_name = "Ashok"
student_email = "ashok@gmail.com"
student_course = "Python"
```

```
def display_student():
    print(student_name)
    print(student_email)
    print(student_course)
```

```
display_student()
```

```
-----
```

```
Note: If we have 100 students, managing data becomes difficult.
```

```
** OOPS helps us to group related data and functions together.
```

```
=====
## Why OOP?
=====
```

```
** Code Re-Usability
```

```
** Code organization
```

```
** Security
```

```
** Easy Maintenance
```

```
** Reduce boiler plate code
```

```
** Building large scale application
```

```
=====
## Main Concepts of OOP
=====
```

- 1) Class
- 2) Object
- 3) Constructor
- 4) Instance Variable
- 5) Instance Method
- 6) Encapsulation
- 7) Inheritance
- 8) Polymorphism
- 9) Abstraction

```
=====
## What is a Class?
=====
```

```
** Class is a blue print or template
```

```
** Class is a plan or model to define properties and behaviours
```

```
    -- properties nothing but variables to store the data
```

```
    -- behaviours nothing but functions to perform the action
```

Ex-1 :

```
Student is a class
```

```
student properties : name, email, phno, course
```

```
student behaviours : study( ), attend_class( ), write_exam ( )
```

Ex-2 :

```
Employee is a class
```

```
employee properties : id, name, salary
```

```
employee behaviours : work(), attend_meetings(), take_leave(), take_salary()
```

```
### Class syntax :
```

```
class ClassName:
    statements
```

```
### Class Example ::
```

```
class Student:
    pass
```

```
### Class Example ::
```

```
class Student:
    id = 100
    name = "Ashok"

    def display(self):
        print(self.name)
        print(self.id)
```

```
=====
What is an Object?
=====
```

**** An object is a real instance created from a class.**

Ex : s1 = Student ()

Note: here s1 is an object of Student class.

```
-----
class Student:
    id = 100
    name = "Ashok"

    def display(self):
        print(self.name)
        print(self.id)
```

```
s1 = Student()
s1.display()
-----
```

```
=====
## What is self?
=====
```

**** self is a reference variable that refers to the current object.**

**** self is used to access variables and methods inside a class.**

**** Whenever we create an object and call a method, Python automatically passes that object to self.**

**** self is not a keyword, but it is a standard naming convention.**

```
-----

class Student:
    id = 100
    name = "Ashok"

    def display(self):
        print(self.name)
        print(self.id)

    def study(self):
        print(self.name, "Student studying...")
```

```
# Creating object
s1 = Student()
```

```
# calling functions using object
```

```
s1.display() ## display(s1)
s1.study() ## study(s1)
```

```
-----
```

Note : When we call `s1.display()` and `s1.study()` , Python internally passes `s1` as `self`.

```
=====
## Constructor
=====
```

**** Constructor is a special method in Python.**

**** Constructor is automatically called when object is created for class.**

**** Constructor is used to initialize object data.**

**** In Python, constructor name is `__init__()`.**

```
=====
## Constructor Syntax
=====
```

```
class ClassName:
    def __init__(self):
        statements
```

```
=====
## Constructor Example
=====
```

```
class Student:
    def __init__(self, name, course):
        self.name = name
        self.course = course

    def display(self):
        print(self.name, "--", self.course)
```

```
s1 = Student("Ashok", "Python")
s1.display()
```

```
s2 = Student("John", "JAVA")
s2.display()
```

```
s3 = Student("GITA", "Ai")
s3.display()
```